

```

/*
 * Adaptive Hollow Reflector - Endpoint Edition
 * Copyright 2026 Gregory Scott Davis
 * Licensed under the Apache License, Version 2.0
 */

use crate::hollow::{EndpointAction, FileHollow, UserPrivilege};
use crate::registry::HollowRegistry;
use crate::telemetry::SocWebhook;
use std::sync::Arc;
use std::time::{SystemTime, UNIX_EPOCH};
use tokio::sync::RwLock;

pub struct AdaptiveHollowReflector {
    registry: Arc<RwLock<HollowRegistry>>,
    soc_webhook: Arc<SocWebhook>,
}

#[derive(Debug, Clone)]
pub struct Decision {
    pub action: EndpointAction,
    pub reason: Vec<String>,
    pub hollow_id: String,
    pub latency_ms: f64,
    pub is_global: bool,
}

impl AdaptiveHollowReflector {
    pub fn new(registry: Arc<RwLock<HollowRegistry>>, soc_webhook: Arc<SocWebhook>) -> Self {
        Self {
            registry,
            soc_webhook,
        }
    }

    pub async fn process_file_event(&self, mut hollow: FileHollow) -> Vec<Decision> {
        let start = SystemTime::now();
        let mut decisions: Vec<Decision> = Vec::new();
        let hollow_id = self.compute_hollow_id(&hollow);

        // 1. Boost risk with peer similarity. Cap at 10.
        hollow.risk_score = (hollow.risk_score + (hollow.peer_risk * 3.0) as u8).min(10);
    }
}

```

```

// HIGHEST PRIORITY: Confirmed threat intel match
// Hash match = no doubt. Kill immediately, don't wait for registry.
if hollow.threat_intel_match {
    decisions.push(self.build_decision(
        EndpointAction::KillTree,
        vec!["threat_intel_match=true".into()],
        &hollow_id,
        start,
        false,
    ));
    return decisions; // Terminal: no further processing needed
}

// SECOND: Global propagation - isolate immediately
// Requires both spread AND danger to avoid FP storms
if hollow.propagation_score > 5 && hollow.risk_score >= 8 {
    // Promote to global invariant for 300s
    self.registry
        .write()
        .await
        .add_invariant(&hollow_id, EndpointAction::IsolateHost, 300, true)
        .await;

    let decision = self.build_decision(
        EndpointAction::IsolateHost,
        vec![
            format!("propagation_score={}", hollow.propagation_score),
            format!("risk_score={}", hollow.risk_score),
            format!("mitre={:?}", hollow.mitre_ttp),
        ],
        &hollow_id,
        start,
        true,
    );

    // Fire SOC webhook async - don't block containment
    let webhook_hollow = hollow.clone();
    let webhook = self.soc_webhook.clone();
    tokio::spawn(async move {
        webhook.send(EndpointAction::SendSocWebhook, &webhook_hollow).await;
    });

    decisions.push(decision);
    return decisions; // Terminal: host is isolated
}

```

```

}

// THIRD: Domain admin compromise - revoke sessions first
// This is a side effect. We still need to handle the file action.
if matches!(hollow.user_privilege, UserPrivilege::DomainAdmin) && hollow.risk_score > 7 {
    decisions.push(self.build_decision(
        EndpointAction::RevokeSession,
        vec![
            "user_privilege=domain_admin".into(),
            format!("risk_score={}", hollow.risk_score),
        ],
        &hollow_id,
        start,
        true, // Revocation is global
    ));
    // Fall through: still need to kill/suspend the process
}

// FOURTH: Clipboard hijack = review, don't break workflow
// Suspend so it stops, but let human decide kill vs release
if hollow.clipboard_monitor {
    decisions.push(self.build_decision(
        EndpointAction::FlagForHumanReview,
        vec!["clipboard_monitor=true".into()],
        &hollow_id,
        start,
        false,
    ));
    return decisions; // Terminal: wait for human
}

// FIFTH: Existing AHR logic for standard cases
let ahr_decision = self.ahr_decision(&hollow, &hollow_id, start).await;
decisions.push(ahr_decision);

decisions
}

async fn ahr_decision(
    &self,
    hollow: &FileHollow,
    hollow_id: &str,
    start: SystemTime,
) -> Decision {

```

```

    // Check for existing invariant first
    if let Some(inv) = self.registry.read().await.check_invariant(hollow_id).await {
        return self.build_decision(inv.action, vec!["invariant_hit".into()], hollow_id, start,
inv.is_global);
    }

    // Predictive: if we see backup tamper, pre-stage invariants for next TTPs
    if hollow.backup_tamper {
        let variants = vec![
            "wmic shadowcopy delete",
            "bcdedit /set {default} recoveryenabled No",
            "wbadmin delete catalog -quiet",
        ];
        for v in variants {
            let variant_id = format!("{:}", hollow_id, v);
            self.registry
                .write()
                .await
                .add_invariant(&variant_id, EndpointAction::SuspendProc, 10, false)
                .await;
        }
    }

    // Standard risk scoring
    let action = match hollow.risk_score {
        9..=10 => EndpointAction::KillTree,
        7..=8 => EndpointAction::SuspendProc,
        4..=6 => EndpointAction::LogOnly,
        _ => EndpointAction::LogOnly,
    };

    let reason = vec![
        format!("risk_score={}", hollow.risk_score),
        format!("entropy_delta={:.2}", hollow.entropy_delta),
        format!("write_velocity={}", hollow.write_velocity),
    ];

    self.build_decision(action, reason, hollow_id, start, false)
}

fn build_decision(
    &self,
    action: EndpointAction,
    reason

```